Google Ads (AdMob) integration

Process + code changes for FixMoney (`com.vikram641.expensetracker`)

Plan for adding Google AdMob ads to FixMoney. Nothing in this doc is implemented yet — it's the reference to work from when actually wiring ads in. Written against the current codebase: single-Activity + Navigation Component, Hilt DI, XML/view-binding layouts, Firebase already integrated (`expenseapp-7b500`).

1. Accounts / console setup (do this first, outside the codebase)

Status: done as of 2026-09-11 — app added manually in AdMob (see note after step 2; the Play Store search couldn't find it since the app isn't public yet).

1. **Create an AdMob account** at admob.google.com — sign in with whichever Google account should own it. Can be the same account as Play Console (`vikram8785`) or a separate one; either works, but write down which one you pick, same as the Firebase-vs-Play-Console account split already in play for this app.
2. **Add the app in AdMob** — "Apps" → "Add app" → search `com.vikram641.expensetracker` . **This will fail while the app is only in Closed testing**: Play only generates a public store-listing page (the one AdMob's search/link-by-URL queries) once an app reaches **Open testing or Production**. Confirmed 2026-09-11 — `https://play.google.com/store/apps/details?id=com.vikram641.expensetracker` 404s publicly even though the closed-testing opt-in link works for enrolled testers. **Fix used**: chose "No" on "Is the app listed on a supported app store?" and added it manually instead (Platform: Android, name: `FixMoney`). This unlocks ad units and local SDK integration/testing immediately. Once the app reaches Open testing/Production, go to **App settings** → **App store details** → **Add** on this same AdMob app entry to link it to the real Play Store listing — no need to recreate anything, ad unit IDs stay the same.
   - **AdMob App ID (real, already created)**: `ca-app-pub-7178541936408370~9464292553` — this is what goes in the manifest's `APPLICATION_ID` meta-data in §5.
   - **Approval status**: "Requires review" (normal for a brand-new app — AdMob reviews before it can serve at full volume; doesn't block integration/testing with test ad unit IDs in the meantime).
3. **Link AdMob to Firebase** (optional but recommended) — since this project already uses Firebase (`expenseapp-7b500`), linking AdMob ↔ Firebase in the AdMob console gives you Firebase Analytics correlation for ad events (impressions/revenue vs. in-app events) for free. Not required for ads to work.
4. **Create ad units** — one per placement you intend to ship (see §3 for placement recommendations). Each ad unit gets its own **Ad unit ID** (`ca-app-pub-xxxxx/yyyy`) — distinct from the **App ID** (`ca-app-pub-xxxxx~zzzz`) you'll put in the manifest. Suggested to start:
   - `Home banner` (Banner)
   - `Post-save interstitial` (Interstitial)
5. **Set up consent (UMP)** — AdMob console → Privacy & messaging → create a GDPR message (EEA/UK) and, if you want, a US states message (CCPA-style). This is not optional for EEA/UK traffic — Google enforces it at

the SDK level (see §5).

6. **Grab your test device ID** once you build a debug APK and run it once — AdMob SDK logs it to Logcat on first `MobileAds.initialize()` call ("Use RequestConfiguration.Builder().setTestDeviceIds(Arrays.asList("XXXX"))"). Use it during development so you never trigger real ad requests/serve real ads to your own test device (Google can suspend accounts for self-clicks/invalid traffic).

2. Play Console declarations (do this before submitting the release that ships ads)

- **App content** → **Ads**: "Does your app contain ads?" → **Yes**. This is a hard requirement — Google detects the AdMob SDK in the binary and will reject/flag the submission if this is left as "No".
- **App content** → **Data safety**: add/confirm data collection for **Advertising ID** and **App activity** (or whatever Play's current wording is for ad-serving data) — shared with Google for advertising purposes, not encrypted-in-transit exemption, user cannot delete via the app. Play's Data safety form UI changes over time; fill it honestly against whatever AdMob's own docs say it collects at the time you do this.
- **Privacy policy** — update `docs/PRIVACY.md` / the live page at `https://vikram641.github.io/My-Expense-App/PRIVACY.html` to disclose use of Google AdMob, the Advertising ID, and third-party ad partners. Google's AdMob program policies require this to be linked from both the Play listing and (ideally) in-app before ads go live.
- This can all happen in the **existing Closed testing - Alpha track** without resetting the 14-day/12-tester production-eligibility clock — that clock tracks continuous tester opt-in on the track, not whether the binary's code changed between releases. A new release with ads wired in just needs to go through review like release 7 did.

3. Ad placement recommendations for this app

Keep density low — this is a finance app; overly aggressive ads are both a bad UX fit and a policy risk (Google's "ad behavior" policies restrict ads that interfere with core app functionality).

Placement	Format	Where
Home screen	Banner	Bottom of `HomeFragment` (`fragment_home.xml`), pinned above the bottom nav, non-intrusive
After saving an expense	Interstitial	`AddExpenseFragment`, shown occasionally (e.g. every 3rd–4th save) after a successful save, before navigating back — never on the *first* save of a session
Expense list	Native (optional, later)	Inline card every N items in `EspenceListFragment`'s RecyclerView — more engineering work (a distinct view-type in the adapter), skip for v1

Explicitly avoid: interstitials on app open/close (against policy for surprising the user), banners inside the receipt scanner camera screen, anything overlapping the bottom nav or FAB.

4. Gradle changes

`gradle/libs.versions.toml`:

```
[versions]
playServicesAds = "23.6.0"
userMessagingPlatform = "3.1.0"

[libraries]
play-services-ads = { module = "com.google.android.gms:play-services-ads", version.ref = "play
user-messaging-platform = { module = "com.google.android.ump:user-messaging-platform", version
```

`app/build.gradle.kts`, `dependencies { }`:

```
// AdMob + consent (UMP)
implementation(libs.play.services.ads)
implementation(libs.user.messaging.platform)
```

Check admob.google.com / Google's Android quickstart for the current version numbers at implementation time — these SDKs update often.

5. AndroidManifest.xml changes

Add the AdMob **App ID** (not an ad unit ID) as `meta-data` inside `<application>`:

```
<meta-data
    android:name="com.google.android.gms.ads.APPLICATION_ID"
    android:value="ca-app-pub-7178541936408370~9464292553" />
```

Missing this crashes the app on `MobileAds.initialize()` with an explicit error, so it's easy to catch if forgotten.

`play-services-ads` auto-merges the `com.google.android.gms.permission.AD_ID` permission into the manifest — no manual addition needed, but it's why the Data safety declaration in §2 is mandatory (Play scans for this permission + the SDK).

6. Code changes

6.1 Initialize the SDK — `ExpenseApp.kt`

In `onCreate()`, after Firebase init (order doesn't strictly matter, but keep it next to the other SDK bootstrapping already there):

```
MobileAds.initialize(this) { /* init status callback, safe to ignore */ }

if (BuildConfig.DEBUG) {
    val config = RequestConfiguration.Builder()
        .setTestDeviceIds(listOf("YOUR_TEST_DEVICE_ID"))
        .build()
    MobileAds.setRequestConfiguration(config)
}
```

6.2 Consent (UMP) — required before any ad request in the EEA/UK

Add a small `AdConsentManager` (`core/util/AdConsentManager.kt`, following the same "one small class per concern" pattern as `TokenManager` / `OnboardingPrefs`), called once from `MainActivity.onCreate()` before the first ad load:

```
val params = ConsentRequestParameters.Builder().build()
val consentInfo = UserMessagingPlatform.getConsentInformation(context)
consentInfo.requestConsentInfoUpdate(
    activity, params,
    { // consent gathered/not required
        if (consentInfo.isConsentFormAvailable) {
            UserMessagingPlatform.loadAndShowConsentFormIfRequired(activity) { /* proceed to l
        } else {
            // proceed to load ads
        }
    },
    { formError -> /* log + proceed anyway, don't block the app on a consent SDK failure */ }
)
```

Only request/load actual ads **after** this callback resolves — don't gate the rest of the app's UI on it.

6.3 New file — `core/ads/AdManager.kt` (Hilt `@Singleton`)

A thin wrapper so ad-loading logic isn't duplicated per-fragment, mirroring how `Repository` centralizes data access:

- `loadBannerAd(adView: AdView)` — builds `AdRequest`, calls `adView.loadAd(...)`.
- `showInterstitial(activity: Activity, onDismissed: () -> Unit)` — pre-loads an `InterstitialAd` lazily, shows it if ready, always calls `onDismissed` (whether an ad showed or not) so callers never block

navigation on ad availability.

- Ad unit IDs: **hardcode test IDs for debug builds** (`ca-app-pub-3940256099942544/...` , Google's public test units), **real IDs for release** — same `BuildConfig.DEBUG` branch pattern as the test-device-ID check above. Never ship a build with test ad unit IDs in `release` , and never test against real ad unit IDs on a personal device (both are against AdMob policy — self-serves count as invalid traffic either way; test IDs exist specifically so you can iterate safely).

6.4 Layout — banner placement

`fragment_home.xml` : add an `AdView` inside the existing root `ConstraintLayout` , constrained to the bottom, above the bottom navigation:

```
<com.google.android.gms.ads.AdView
    android:id="@+id/adViewHome"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:adSize="BANNER"
    app:adUnitId="@string/admob_banner_home"
    app:layout_constraintBottom_toTopOf="@id/bottomNav"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Put the ad unit ID in `strings.xml` per the existing pattern of string-resource externalization used elsewhere in the app, not inlined.

`HomeFragment.kt` : call `adManager.loadBannerAd(binding.adViewHome)` in `onViewCreated()` , and forward lifecycle calls (`adView.resume()` / `pause()` / `destroy()`) from the fragment's own `onResume()` / `onPause()` / `onDestroyView()` — `AdView` needs these forwarded or it leaks/keeps polling in the background.

6.5 Interstitial trigger — `AddExpenseFragment.kt`

After a successful save (wherever the fragment currently pops the back stack / navigates home on save success), gate behind a simple counter in a small prefs class (or reuse `OnboardingPrefs` 's pattern) so it's not shown on *every* save:

```
if (expenseSaveCounter.incrementAndGet() % 4 == 0) {
    adManager.showInterstitial(requireActivity()) { findNavController().popBackStack() }
} else {
    findNavController().popBackStack()
}
```

7. Testing checklist before submitting a release

1. `RequestConfiguration.setTestDeviceIds(...)` set for debug builds, verified in Logcat that requests are tagged test.
2. Banner renders on `HomeFragment` without shifting/covering existing content (budget card, bottom nav) at both small and large device sizes.
3. Interstitial shows only after the Nth save, never on first launch, never blocks a save that failed.
4. Consent form (UMP) actually appears when testing with a VPN/device region set to an EEA country; app still functions normally if consent is declined (ads just won't personalize, or won't show at all depending on the form's configured behavior).
5. `./gradlew bundleRelease` succeeds with **real** ad unit IDs wired for the release build type — grep the built AAB/manifest to confirm no `ca-app-pub-3940256099942544` (Google's shared test publisher ID) leaked into a release build.
6. Play Console → App content → Ads set to "Yes", Data safety updated, before submitting.
7. Pull the actual Play-Store-distributed APK after rollout and confirm ads render there too — same "verify against the distributed artifact" discipline as every other release for this app (Play's own Automatic-protection/App Bundle processing has bitten this project before; see `CLAUDE.md`'s Pairip section).

8. Policy notes worth keeping in mind

- Never tap/click your own ads, on any device, ever — AdMob detects this as invalid traffic and can suspend the whole account (not just this app).
- Don't place ads that could be confused for app UI (e.g. a "Continue"-styled native ad button).
- Interstitials must have a clearly visible close button and can't be shown so frequently that they're the dominant UX.
- Since this is a personal-finance app, don't run ad networks/mediation that push loan/credit-adjacent ads without checking they fit Google's finance-app ad content policies — a mismatch here is the kind of thing that gets a whole account flagged, not just one ad unit.